

1. Tempo disponibile 120 minuti.
2. Non è possibile consultare appunti, slide, libri, persone, siti web, ecc.
3. Scrivere in modo leggibile, su ogni foglio, nome, cognome e numero di matricola.
4. Le soluzioni agli esercizi che richiedono di progettare un algoritmo devono:
 - spiegare a parole l'algoritmo (se utile, anche con l'aiuto di esempi o disegni),
 - fornire e commentare lo pseudo-codice (indicando il significato delle variabili),
 - calcolare la complessità (con tutti i passaggi matematici necessari),
 - se l'esercizio ammette più soluzioni, a soluzioni computazionalmente più efficienti e/o concettualmente più semplici sono assegnati punteggi maggiori.
5. Nel caso di utilizzo di tecniche differenti da quelle descritte nel materiale didattico del corso, come ad esempio tecniche suggerite da intelligenze artificiali generative tipo ChatGPT, dovete giustificare la vostra scelta commentando le differenze ed i vantaggi (utilizzando, ove applicabile, l'analisi dei costi computazionali) rispetto all'uso delle tecniche descritte a lezione.

IMPORTANTE: Risolvere gli esercizi 1–2 e gli esercizi 3–4 su fogli separati. Infatti, al termine, dovete consegnare gli esercizi 1–2 separatamente dagli esercizi 3–4.

1. Calcolare la complessità $T(n)$ del seguente algoritmo MYSTERY1:

Algorithm 1: MYSTERY1(INT n) $\rightarrow$ INT

```

 $x = 0$ 
for  $i = 1, \dots, n$  do
   $x = x + \text{MYSTERY2}(\log i^{100})$ 
return  $x$ 

function MYSTERY2(INT  $n$ )  $\rightarrow$  INT
if  $n \leq 0$  then
  return 1
else
  return MYSTERY2( $n/2$ ) + MYSTERY2( $n/2$ ) +  $n^2 + n$ 

```

Soluzione. Analizziamo prima il costo di MYSTERY2, che esegue due chiamate ricorsive con input ridotto ad $1/2$ del valore originale. Le rimanenti operazioni in MYSTERY2 hanno costo costante. L'equazione di ricorrenza di MYSTERY2:

$$T'(n) = \begin{cases} 1 & n \leq 1 \\ 2T'(n/2) + 1 & n > 1 \end{cases}$$

può essere risolta con il Master Theorem:

$$\alpha = \log_2 2 = 1 > 0 = \beta \Rightarrow T'(n) = \Theta(n^\alpha) = \Theta(n)$$

Il costo di MYSTERY1 dipende dal suo unico ciclo for. Tale ciclo viene eseguito n volte, per i che va da 1 ad n . Ad ogni iterazione viene chiamata la funzione MYSTERY2 con input $\log i^{100}$. Poiché il costo di MYSTERY2 è lineare rispetto al proprio input, il costo della chiamata alla generica iterazione è:

$$\Theta(\log i^{100}) = \Theta(100 \log i) = \Theta(\log i)$$

Il costo totale del ciclo **for** è quindi:

$$\Theta(\log 1) + \cdots + \Theta(\log n) = \Theta\left(\sum_{i=1}^n \log i\right) = \Theta\left(\log \prod_{i=1}^n i\right) = \Theta(\log n!) = \Theta(n \log n)$$

Pertanto, la complessità dell'algoritmo MYSTERY1 è $\Theta(n \log n)$.

2. Progettare un algoritmo che, date in input due liste concatenate semplici, L1 e L2, contenenti interi ordinati in ordine crescente, ritorni il numero di elementi di L1 che non compaiono in L2, senza modificare nessuna delle due liste. Ad esempio, se

$$L1 = [1, 1, 2, 2, 3, 5, 10] \text{ e } L2 = [1, 3, 3, 7]$$

allora l'algoritmo dovrà restituire il valore 4, poiché gli elementi di L1 che non compaiono in L2 sono 2, 2, 5, 10.

Analizzare il costo della soluzione proposta nel caso pessimo. Il punteggio massimo verrà assegnato a soluzioni con costo pessimo lineare rispetto al numero totale di nodi presenti nelle due liste.

Soluzione.

Una procedura ricorsiva consente di risolvere il problema in modo efficiente. Ad ogni passo si confrontano gli elementi correnti delle due liste e si decide se incrementare il conteggio di elementi di L1 assenti in L2 e se avanzare in L1 o in L2.

Algorithm 2: DIFFCOUNT(NODE L1, NODE L2) → INT

```

if L1 == NIL then
  | return 0
else if L2 == NIL or L1.key < L2.key then
  | return 1 + DIFFCOUNT(L1.next, L2)
else if L1.key > L2.key then
  | return DIFFCOUNT(L1, L2.next)
else
  | return DIFFCOUNT(L1.next, L2)

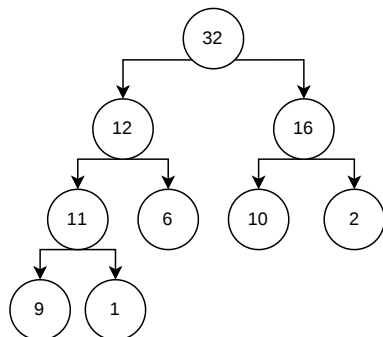
```

Assumiamo che $|L1| = m$ e $|L2| = n$. Nel caso pessimo, l'algoritmo visita tutti i nodi di entrambe le liste. Poiché ad ogni chiamata ricorsiva avanza almeno uno dei due puntatori, il numero totale di chiamate è al più $m + n$. Ne consegue che il costo nel caso pessimo è: $\Theta(m + n)$.

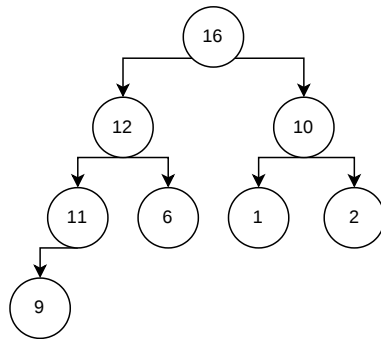
Una situazione in cui l'algoritmo è costretto a visitare tutti i nodi si verifica quando gli elementi delle due liste sono ordinati e intercalati. Ad esempio, $L1 = [1, 3, 5, 7, 9, \dots, 2k + 1]$ e $L2 = [0, 2, 4, 6, 8, 10, \dots, 2k]$. In questo caso, gli elementi delle due liste si alternano e nessun elemento compare in entrambe le liste. Di conseguenza, ad ogni passo l'algoritmo avanza di un solo nodo ed è costretto a visitare tutti i nodi di entrambe le liste.

3. Si consideri l'array-max-heap binario $[32, 12, 16, 11, 6, 10, 2, 9, 1]$. Descrivere come cambia l'array in seguito all'esecuzione di due operazioni *deleteMax*.

Soluzione. La rappresentazione ad albero binario dell'array risulta essere la seguente:

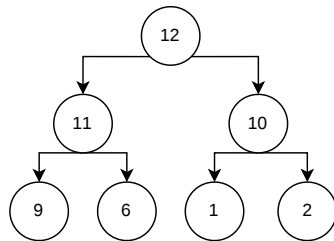


Le prima esecuzione dell'operazione *deleteMax* trasforma l'albero nel seguente modo:



di conseguenza l'array diviene [16, 12, 10, 11, 6, 1, 2, 9].

La seconda esecuzione trasforma l'albero nel seguente modo:



di conseguenza l'array diviene [12, 11, 10, 9, 6, 1, 2].

4. Si consideri il seguente problema di soddisfacimento di vincoli: date n variabili $x_1, \dots, x_n$ e m disequazioni del tipo $x_{i_1} < x_{j_1}, \dots, x_{i_m} < x_{j_m}$ bisogna verificare se esiste un assegnamento di numeri interi alle variabili che soddisfa tutte le disequazioni. Ad esempio, l'istanza del problema con 3 variabili x_1, x_2, x_3 e disequazioni $x_1 < x_3$, $x_3 < x_2$ e $x_1 < x_2$ è soddisfacibile (ad esempio dall'assegnamento $x_1 = 1$, $x_2 = 3$ e $x_3 = 2$). Invece, l'istanza del problema con 3 variabili x_1, x_2, x_3 e disequazioni $x_1 < x_3$, $x_3 < x_2$ e $x_2 < x_1$ non è soddisfacibile.

Progettare un algoritmo che risolve tale problema ricevendo in input il numero n di variabili e due array $i[1..m]$ e $j[1..m]$ che rappresentano le m disequazioni $x_{i[1]} < x_{j[1]}, \dots, x_{i[m]} < x_{j[m]}$. Ad esempio, la prima istanza del problema riportata sopra viene descritta dall'input $n = 3$, $i = [1, 3, 1]$ e $j = [3, 2, 2]$, mentre la seconda istanza corrisponde all'input $n = 3$, $i = [1, 3, 2]$ e $j = [3, 2, 1]$.

Soluzione. Il problema può essere risolto trasformando l'input in un grafo orientato che contiene un vertice x_i per ogni variabile e un arco da x_i a x_j per ogni disequazione $x_i < x_j$. L'istanza in input risulta essere soddisfacibile se e solo se tale grafo risulta essere aciclico; infatti, un sistema di disequazioni risulta essere soddisfacibile se e solo se non esiste una sequenza di disequazioni che, per transitività, implicherebbe che una variabile deve essere minore di se stessa. Per verificare la presenza di cicli si utilizza una visita DFS che si interrompe segnalando la presenza del ciclo nel caso in cui si trovi un arco all'indietro. Se si termina la visita senza trovare archi all'indietro, allora il grafo è aciclico.

Tale algoritmo è descritto in pseudocodice come Algoritmo 3. Se si assume implementazione tramite liste di adiacenza la creazione del grafo richiede tempo $\Theta(n + m)$ (in quanto le operazioni *addVertex* e *addEdge* hanno costo costante) mentre la parte di verifica del ciclo ha costo $O(n + m)$ (costo nel caso pessimo che si verifica quando non sono presenti cicli e quindi si rende necessaria l'intera visita). Complessivamente l'algoritmo ha costo computazionale $\Theta(n + m)$.

Algorithm 3: DISEQUAZIONISODDISFACIBILI(INT n , INT $i[1..m]$, INT $j[1..m]$) $\rightarrow$ BOOL

```
// Creazione grafo
GRAPH  $g = \text{new GRAPH}()$ 
VERTEX  $V[1..n]$ 
for  $l \in [1..n]$  do
   $V[l] = g.\text{addVertex}()$ 
for  $k \in [1..m]$  do
   $g.\text{addEdge}(V[i[k]], V[j[k]])$ 
// Esecuzione DFS: se la DFS trova un ciclo si restituisce false, altrimenti true
for  $v \in V[1..n]$  do
   $v.\text{mark} \leftarrow \text{white}$ 
for  $v \in V[1..n]$  do
  if ( $v.\text{mark} = \text{white}$ ) then
    if DFSVISIT( $v$ ) then
      return false
return true

// DFS che restituisce true se trova un ciclo
DFSVISIT(VERTEX  $u$ )  $\rightarrow$  BOOL
 $u.\text{mark} \leftarrow \text{grey}$ 
for  $v \in u.\text{adjacents}$  do
  if  $v.\text{mark} = \text{grey}$  then
    return true
  if ( $v.\text{mark} = \text{white}$ ) then
    if DFSVISIT( $v$ ) then
      return true
 $u.\text{mark} \leftarrow \text{black}$ 
return false
```
